```
      def method_body(method_name, const_name)
        <<-EOMETHOD
        def #{method_name}
          name = ::ActiveRecord::AttributeMethods::AttrNames::ATTR_#{const_name}
          read_attribute(name) { |n| missing_attribute(n, caller) }
        end
        EOMETHOD
      end
    }.new
```

ReaderMethodCache is an instance of an anonymous class—a subclass of AttributeMethodCache. This class defines a single method that returns a *String of Code (141)*. (If you're perplexed by the call to Class.new, take a look back at *Quiz: Class Taboo, on page 112*. If you don't understand the EOMETHOD lines, read about "here documents" in *The REST Client Example, on page 141*.)

Let's leave ReaderMethodCache for a moment and move to the definition of its superclass AttributeMethodCache:

gems/activerecord-4.1.0/lib/active_record/attribute_methods.rb

```
module ActiveRecord
  module AttributeMethods
    AttrNames = Module.new {
      def self.set_name_cache(name, value)
        const_name = "ATTR_#{name}"
        unless const_defined? const_name
          const_set const_name, value.dup.freeze
        end
      end
    }

    class AttributeMethodCache
      def initialize
        @module = Module.new
        @method_cache = ThreadSafe::Cache.new
      end
      def [](name)
        @method_cache.compute_if_absent(name) do
          safe_name = name.unpack('h*').first
          temp_method = "__temp__#{safe_name}"
          ActiveRecord::AttributeMethods::AttrNames.set_name_cache safe_name, name
          @module.module_eval method_body(temp_method, safe_name),
                              __FILE__, __LINE__
          @module.instance_method temp_method
        end
      end

      private
      def method_body; raise NotImplementedError; end
    end
```